ahow to Create an App Registration in Azure Microsoft Entra ID for Terraform OIDC. using this I need to deploy the resources in AWS

To deploy AWS resources using Terraform with **Azure Microsoft Entra ID (formerly Azure AD) via OIDC**, you're essentially setting up **federated identity between Azure and AWS** so Terraform can authenticate without storing secrets.

I'll walk you through it clearly 👇

🧭 Overview of What You're Building

- Create an **App Registration in Azure Entra ID**
- Configure **OIDC federation**
- Create an **AWS IAM OIDC Identity Provider**
- Create an **IAM Role in AWS**
- Let Terraform assume that role using Azure-issued tokens

🟦 Step 1: Create App Registration in Azure Entra ID

📍 Navigate

Go to:
👉 Microsoft Entra ID
→ **App registrations** → **New registration**

🛠️ Configure

- **Name**: `terraform-oidc-app`
- **Supported account types**: Single tenant
- Redirect URI: (leave empty)

Click **Register**

🔑 Step 2: Capture Important Values

After creation, note:

- **Application (client) ID**
- **Directory (tenant) ID**

You'll need these later in AWS and Terraform.

🔐 Step 3: Add Federated Credentials (OIDC)

Go to:
👉 App → **Certificates & secrets** → **Federated credentials**

Click **Add credential**

Example Configuration

- **Issuer**: `https://login.microsoftonline.com/<TENANT_ID>/v2.0`
- **Subject**: depends on your use case (e.g. GitHub, workload identity, etc.)
- **Audience**: `api://AzureADTokenExchange`

👉 This defines *who can request tokens*

🧩 Step 4: Create AWS OIDC Identity Provider

Go to:
👉 AWS IAM
→ Identity providers → Add provider

Configure:

- **Provider type**: OpenID Connect
- **Provider URL**:

```
https://login.microsoftonline.com/<TENANT_ID>/v2.0
```

- **Audience**:

```
api://AzureADTokenExchange
```

🔐 Step 5: Create IAM Role for OIDC

Create a role with **trusted entity = OIDC provider**

Trust Policy Example

ChatGPT

```
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/login.microsoftonli
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "login.microsoftonline.com/<TENANT_ID>/v2.0:aud": "api://AzureADTokenEx
        }
      }
    }
  ]
}
```

Attach required permissions (e.g., `AdministratorAccess` or least privilege).

⚙️ Step 6: Configure Terraform

Terraform will use the OIDC token to assume the role.

Example AWS Provider

hcl

```hcl
provider "aws" {
  region = "us-east-1"

  assume_role_with_web_identity {
    role_arn           = "arn:aws:iam::<ACCOUNT_ID>:role/<ROLE_NAME>"
    web_identity_token = file("/path/to/token")
```

```
  }
}
```

🔄 Step 7: Generate Azure OIDC Token

You need to request a token from Azure:

Example using Azure CLI:

Bash

```bash
az account get-access-token \
  --resource api://AzureADTokenExchange \
  --query accessToken \
  --output text > token.jwt
```

✅ How It Works (Simple Flow)

1. Azure Entra ID issues OIDC token
2. Terraform uses that token
3. AWS validates it via OIDC provider
4. AWS allows role assumption
5. Terraform deploys resources 🚀

⚠️ Common Pitfalls

- ❌ Wrong **audience** (`api://AzureADTokenExchange` must match)
- ❌ Incorrect **issuer URL**
- ❌ Missing trust policy condition
- ❌ Token expiration (short-lived!)

💡 Pro Tip

Use:

- GitHub Actions / Azure DevOps OIDC instead of manual tokens

- Short-lived credentials (more secure 🔐)

If you want, I can:

- Give you a **complete Terraform working repo**
- Show **GitHub Actions → Azure → AWS setup**
- Help debug your current config

Just tell me 👍